Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 2

Implementation of a Multi-Layer Perceptron (MLP) for Multi-Class Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

Implement an MLP using TensorFlow/Keras on the Fashion-MNIST dataset. Learn image preprocessing, flattening, model construction, training, evaluation, and automated hyperparameter optimization.

2. Learning Outcomes

Students will be able to:

1. Explain the architecture of an MLP.
2. Preprocess image datasets.
3. Build and train an MLP.
4. Evaluate classification performance.
5. Perform automated hyperparameter optimization.
6. Recommend the best model configuration.

3. Background Theory

An MLP consists of an input layer, hidden layers and an output layer.

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}, \qquad a^{(l)} = f(z^{(l)})$$

Output layer:

$$\hat{y} = \text{Softmax}(z)$$

Loss:

$$L = -\sum_i y_i \log(\hat{y}_i)$$

Recommended architecture:

$$784 \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(10, \text{Softmax})$$

4. Dataset

Dataset: Fashion-MNIST

Training Images: 60,000

Testing Images: 10,000

Classes: 10

Image Size: 28×28

5. Image Preprocessing

Fashion-MNIST images are stored as 28×28 matrices. Dense layers require a one-dimensional feature vector.

$$28 \times 28 = 784$$

Example:

$$\begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix} \rightarrow [10, 20, 30, 40]$$

Perform the following preprocessing:

1. Load Fashion-MNIST.
2. Display sample images.
3. Flatten images.
4. Normalize pixels to [0,1].
5. Convert labels into one-hot vectors.

6. Experimental Procedure

Task 1: Dataset Exploration

- Load Fashion-MNIST.
- Print dataset dimensions.
- Display ten sample images.
- Plot class distribution.

Expected Output: Dataset summary and sample images.

Task 2: Data Preprocessing

- Flatten images.
- Normalize pixel values.
- Print tensor shapes before and after preprocessing.

Task 3: Model Construction

Construct an MLP with

- Input layer (784)
- Dense(128, ReLU)
- Dense(64, ReLU)
- Dense(10, Softmax)

Display `model.summary()`.

Task 4: Model Training

Compile using

- Optimizer: Adam
- Loss: Categorical Cross Entropy
- Metric: Accuracy

Train for 20 epochs with batch size 32.

Task 5: Model Evaluation

Compute

- Accuracy

- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

7. Hyperparameter Optimization

Instead of manually selecting hyperparameters, students shall perform automated hyperparameter optimization using either **GridSearchCV** or **RandomizedSearchCV** (recommended) together with the SciKeras wrapper.

Optimize the following search space.

Hyperparameter	Candidate Values
Hidden Layers	1, 2, 3
Hidden Neurons	32, 64, 128, 256
Learning Rate	0.1, 0.01, 0.001
Batch Size	16, 32, 64, 128
Epochs	10, 20, 30
Optimizer	SGD, Adam, RMSProp
Activation Function	ReLU, Tanh, Sigmoid
Dropout Rate (Optional)	0.0, 0.2, 0.5

Tasks

1. Build a baseline MLP model.
2. Define the hyperparameter search space.
3. Perform Grid Search or Randomized Search using 5-fold cross-validation.
4. Record the best hyperparameter combination.
5. Retrain the model using the optimized hyperparameters.
6. Evaluate the optimized model on the testing dataset.
7. Compare the optimized model with the baseline model.

8. Source Code

GitHub Repository:
`https://github.com/Teena-Pranava/Deep-Learning-Lab`

9. Mandatory Plots

1. Sample Images
2. Class Distribution
3. Training Accuracy vs Epoch
4. Validation Accuracy vs Epoch
5. Training Loss vs Epoch
6. Validation Loss vs Epoch
7. Confusion Matrix

8. Hyperparameter Search Results
9. Best Model Accuracy Comparison

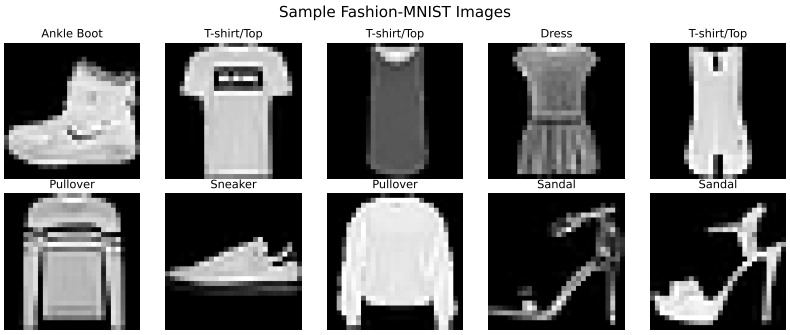

Figure 1: Sample Images

The smaple images shoe different types of clothing items present in Fashion-MNIST dataset. The images are grayscale and have size of 28*28 pixels.

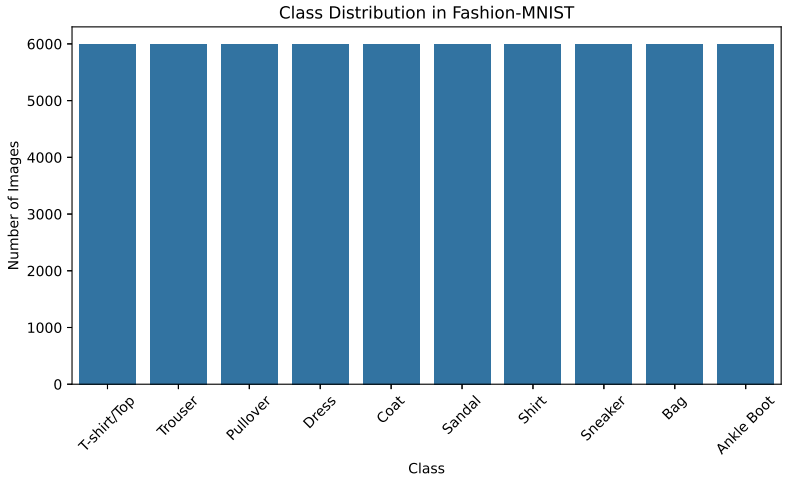

Figure 2: Class Distribution

The class distribution graph shows that all the classes have aomost same number of images. Which means the dataset is balanced and the model gets a fair amount of training data for every class.

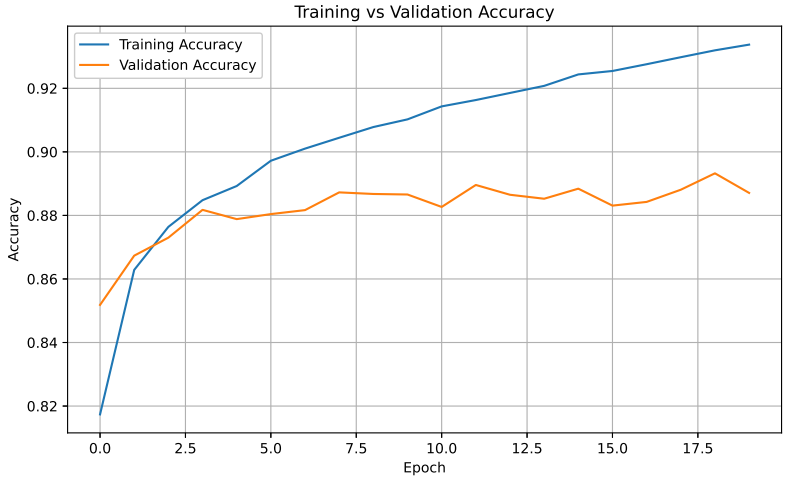

Figure 3: Training Accuracy vs Epoch

The training accuracy increases with each epoch and reaches a stable value.The improvementdecomes samller after many epochs,this indicates that the model has reached convergence.

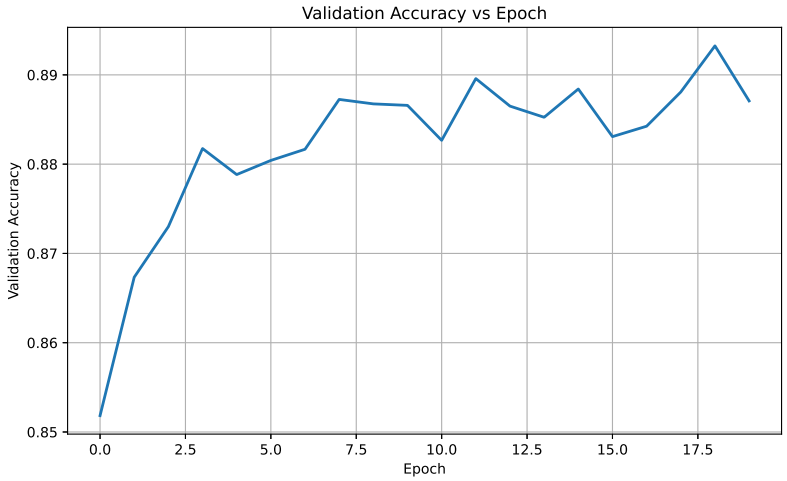

Figure 4: Validation Accuracy vs Epoch

The validation accuracy also increases with training and remains close to training accuracy.This indicates that model performs well on unseen data and there is less overfitting.

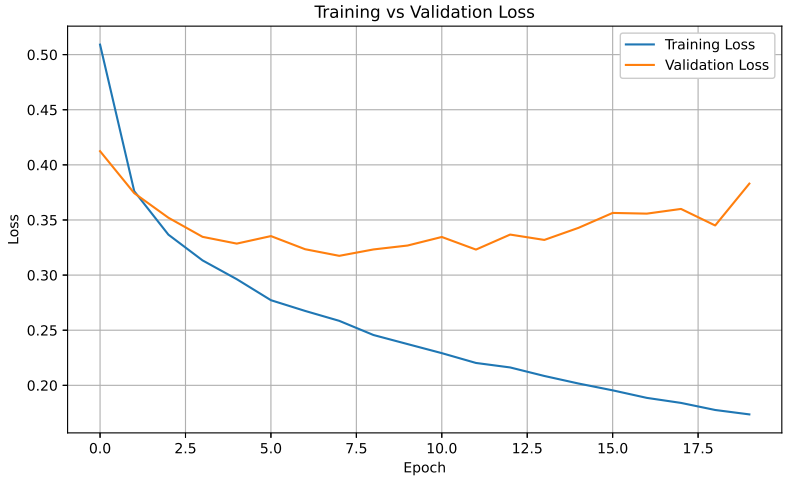

Figure 5: Training Loss vs Epoch

The training loss decreases continuously during training. The loss stabilizes after a few epochs, showing that the model has learned the important features from the dataset.

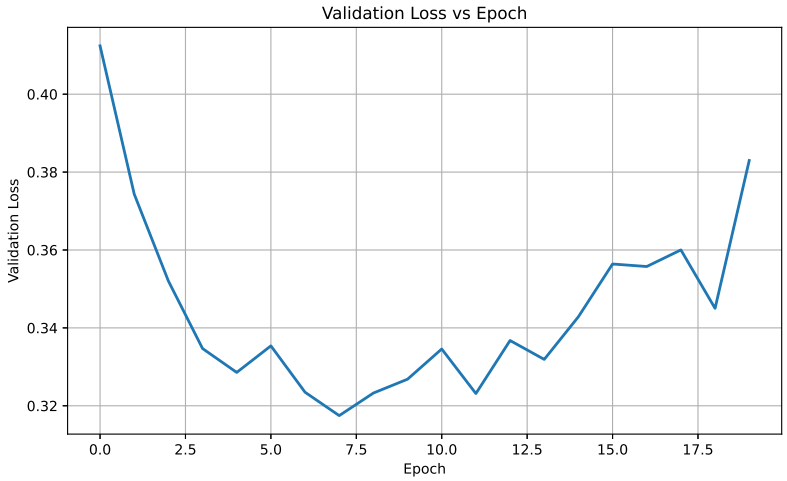

Figure 6: Validation Loss vs Epoch

The validation loss decreases during training and remains close to the training loss. This indicates that the model is able to perform well on validation data.

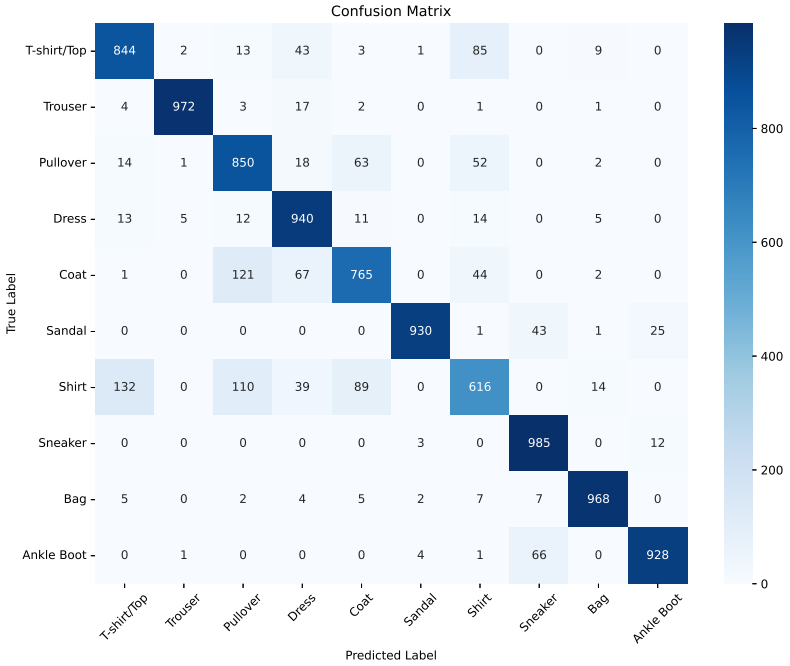

Figure 7: Confusion Matrix

The confusion matrix shows that most images are classified correctly, as seen from the high values along the diagonal. This indicates that the MLP model performs well on the Fashion-MNIST dataset.

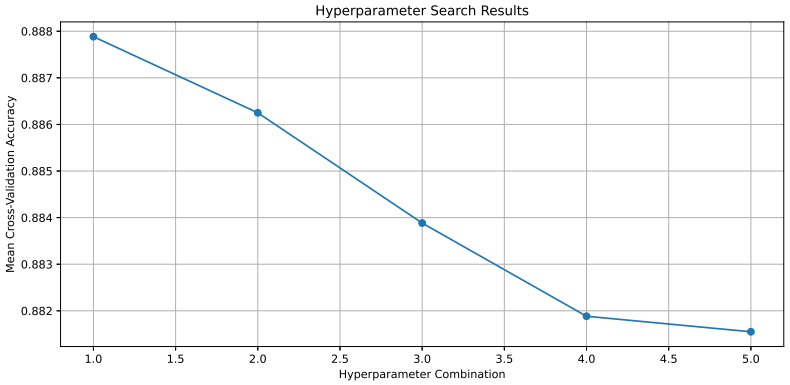

Figure 8: Hyperparameter Search Results

The hyperparameter search compares different combinations of model settings. Some combinations achieved higher cross-validation accuracy than others, showing that the choice of hyperparameters affects the model's performance. The best combination was selected for the final optimized model.

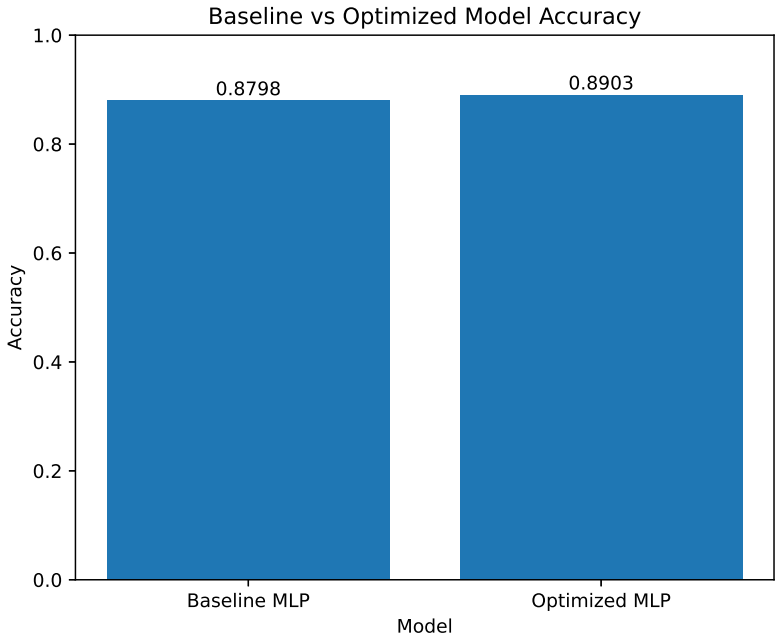

Figure 9: Best Model Accuracy Comparison

The optimized model achieved slightly higher accuracy than the baseline model. This shows that tuning the hyperparameters helped improve the classification performance.

Best Hyperparameters

Hyperparameter	**Best Value**
Hidden Layers	2
Hidden Neurons	128, 64
Learning Rate	0.001
Batch Size	64
Optimizer	RMSprop
Activation Function	ReLU
Epochs	20
Dropout	0.0
Cross-validation Accuracy	0.8879
Testing Accuracy	0.8903

Performance Comparison

Metric	Baseline	Optimized
Accuracy	0.8798	0.8903
Precision	0.8800	0.8911
Recall	0.8798	0.8903
F1-score	0.8783	0.8904
Training Time (s)	153.17	83.38

10. Discussion

1. Which optimization method was used?

Randomized Search with 5-fold cross-validation was used to find the best combination of hyperparameters. It tested different combinations and selected the one with the highest cross-validation accuracy.

2. Which hyperparameters were selected?

The best model used 2 hidden layers with 128 and 64 neurons, RMSprop optimizer, ReLU activation function, batch size of 64, and 20 epochs. The learning rate used was 0.001, which is the default value of the RMSprop optimizer.

3. Did optimization improve performance?

Yes, The optimized model performed slightly better than the baseline model. The testing accuracy improved from 87.98% to 89.03%, precision, recall and F1-score also increased.

4. Which hyperparameter had the greatest impact?

The optimizer and the number of hidden neurons had the greatest impact on the model's performance. Using the RMSprop optimizer with 128 and 64 neurons helped the model learn better and achieve higher accuracy.

5. Compare Grid Search and Randomized Search.

Grid Search checks every possible combination of hyperparameters, so it takes more time. Randomized Search tests only a selected number of combinations, making it faster while still giving good results. Randomized Search is used in this experiment because it is more efficient.

6. Recommend the best MLP configuration.

The recommended MLP configuration is an input layer of 784 neurons, two hidden layers with 128 and 64 neurons, ReLU activation, RMSprop optimizer, batch size of 64, 20 epochs, and a Softmax output layer with 10 neurons. This configuration gave the best performance in the experiment.

11. Additional Task

Question

Code the perceptron learning algorithm for the XOR gate using Python. Show the weights after each update. Plot the decision boundary after each weight update using Python's built-in packages. Analyze and identify the pattern that prevents the algorithm from converging. (K4)

Weight Updates

Table 1: Weight Updates During MLP Training for XOR Gate

Update	$W1_{11}$	$W1_{12}$	$W1_{21}$	$W1_{22}$	$b1_1$	$b1_2$	$W2_{11}$	$W2_{21}$	b2	Loss
1	-0.2451	0.8981	0.4657	0.1953	-0.6820	-0.6919	-0.8926	0.7179	0.1745	0.254485
2	-0.2398	0.8953	0.4669	0.1936	-0.6770	-0.6950	-0.8998	0.7058	0.1516	0.253427
3	-0.2347	0.8928	0.4678	0.1922	-0.6727	-0.6974	-0.9056	0.6955	0.1327	0.252695
4	-0.2301	0.8907	0.4683	0.1911	-0.6691	-0.6991	-0.9102	0.6869	0.1172	0.252189
5	-0.2257	0.8889	0.4686	0.1902	-0.6661	-0.7004	-0.9139	0.6796	0.1045	0.251841
6	-0.2215	0.8872	0.4687	0.1895	-0.6635	-0.7013	-0.9167	0.6734	0.0941	0.251600
7	-0.2176	0.8858	0.4686	0.1889	-0.6613	-0.7019	-0.9189	0.6681	0.0857	0.251433
8	-0.2138	0.8845	0.4683	0.1885	-0.6594	-0.7023	-0.9205	0.6635	0.0788	0.251315
9	-0.2102	0.8834	0.4680	0.1881	-0.6579	-0.7024	-0.9217	0.6595	0.0732	0.251231
10	-0.2067	0.8823	0.4675	0.1878	-0.6565	-0.7024	-0.9224	0.6561	0.0686	0.251170
11	-0.2033	0.8813	0.4669	0.1875	-0.6553	-0.7023	-0.9229	0.6531	0.0650	0.251125
12	-0.2001	0.8804	0.4663	0.1873	-0.6543	-0.7020	-0.9231	0.6504	0.0621	0.251091
13	-0.1969	0.8796	0.4656	0.1872	-0.6535	-0.7017	-0.9232	0.6480	0.0597	0.251063
14	-0.1937	0.8788	0.4649	0.1871	-0.6527	-0.7013	-0.9230	0.6458	0.0579	0.251041
15	-0.1906	0.8780	0.4642	0.1870	-0.6520	-0.7008	-0.9228	0.6439	0.0564	0.251021
16	-0.1876	0.8773	0.4634	0.1869	-0.6514	-0.7003	-0.9224	0.6420	0.0553	0.251004
17	-0.1846	0.8766	0.4626	0.1869	-0.6509	-0.6997	-0.9219	0.6404	0.0545	0.250989
18	-0.1817	0.8759	0.4618	0.1868	-0.6504	-0.6991	-0.9214	0.6388	0.0538	0.250975
19	-0.1788	0.8753	0.4610	0.1868	-0.6499	-0.6985	-0.9208	0.6373	0.0534	0.250962
20	-0.1759	0.8747	0.4602	0.1868	-0.6495	-0.6979	-0.9201	0.6359	0.0531	0.250949

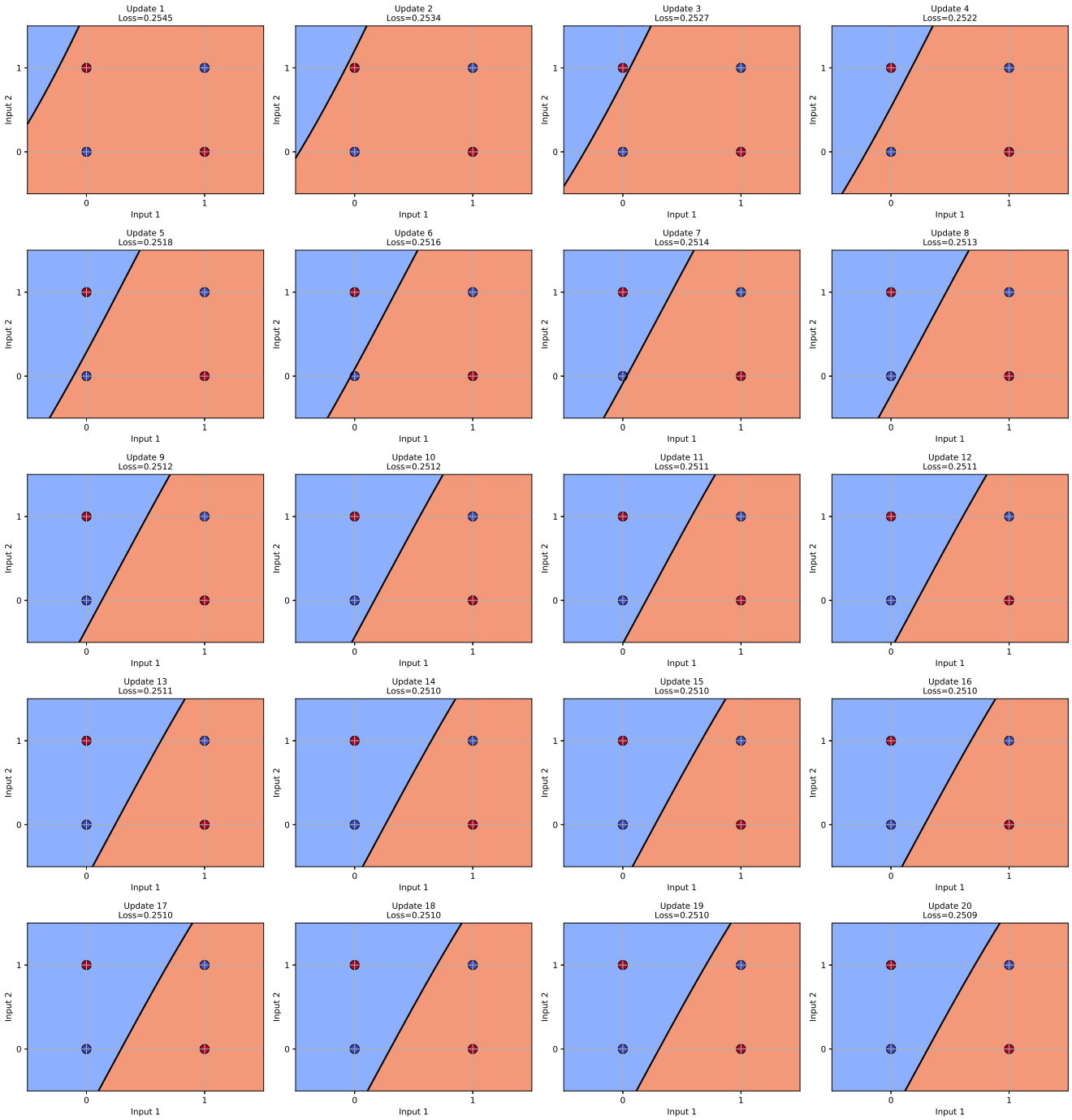

Figure 10: Decision boundary of the XOR gate after each weight update using a Multi-Layer Perceptron.

Final Predictions

Input	Predicted Output
(0,0)	0.487
(0,1)	0.469
(1,0)	0.530
(1,1)	0.513

Analysis

The XOR gate is not linearly seperable, so a single decision boundary cannot correctly classify all the input points.During training, the weights keep changing, but the network doesn't reach the expected outputs. The predictions remain close to 0.5 and the loss decreases only slightly, including that the algorithm has not converged. More training epochs or a properly trained MLP are required to learn the XOR function.

12. Report Contents

Include Objective, Theory, Dataset, Experimental Procedure, Source Code, Hyperparameter Optimization, Results, Plots with Inference, Discussion, Conclusion and References.

13. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. Fashion-MNIST Dataset.
5. TensorFlow/Keras Documentation.